AWS SERIES · MODULES 6–7

SQS, SNS and DynamoDB Streams for Beginners

How AWS services tell each other that something happened — without calling each other directly. A DynamoDB stream reports changes in a table, SNS tells many receivers at once, and SQS keeps work safely until a worker is ready. Explained with one example: a new employee joins.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Services: Amazon DynamoDB Streams · Amazon SNS (Simple Notification Service) · Amazon SQS (Simple Queue Service) · AWS Lambda
Before this module: IAM for Beginners and the lab "Lambda loads a file from S3 into DynamoDB". Next: the lab "DynamoDB Streams → SNS → SQS". Region `us-east-1`

1. Why services pass messages
2. Three tools, one picture
3. DynamoDB Streams: the table's change log
4. How a Lambda reads a stream
5. SNS: one message, many receivers
6. What a subscriber receives
7. SQS: a to-do list that waits
8. How a Lambda works through a queue
9. SNS + SQS together: fan-out
10. The permissions this needs
11. When to use which
12. The lab in one picture
13. What you can skip for now
14. Words to know
15. Key takeaways

1 Why services pass messages

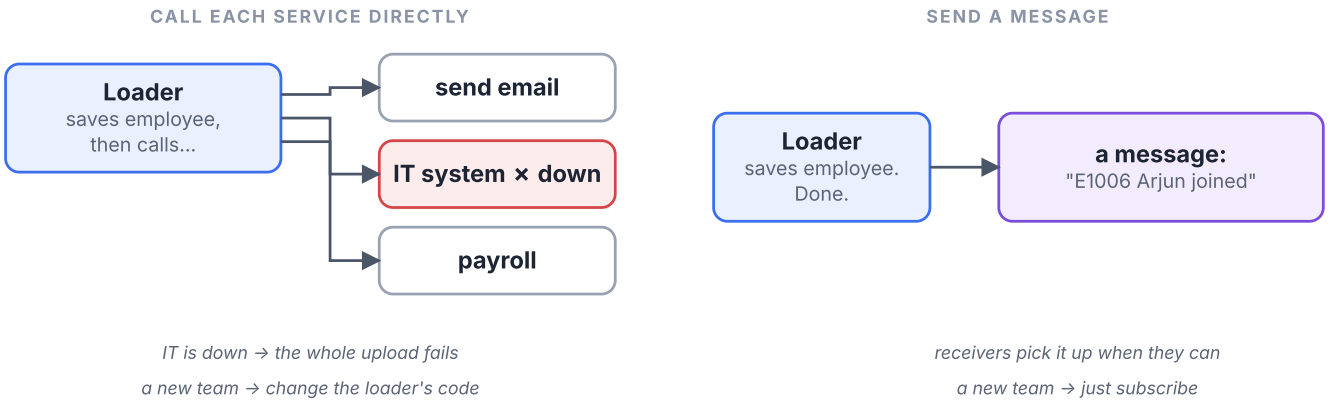


Figure 1. Calling everything directly ties the services together. A message in the middle lets each part work on its own.

When a new employee is saved, several teams need to know: HR sends a welcome email, IT prepares a laptop, payroll adds a salary record. If the loader calls each of them itself, one slow or broken service breaks the upload, and every new team means changing the loader's code.

Instead, the loader only saves the employee. A **message** — "E1006 Arjun joined" — carries the news to whoever needs it. The sender doesn't know or care who receives it. This is called **decoupling**.

> Today you meet three AWS services that carry messages. Each does one job.

2 Three tools, one picture



Figure 2. Stream = what changed. SNS = tell everyone now. SQS = keep the work until it is done.

Service	Think of it as	Messages are…	In our example
DynamoDB Streams	The table's change log	created by DynamoDB itself on every change	"E1006 was inserted"
SNS	A loudspeaker	**pushed** at once to every subscriber; SNS does not keep them	email to HR + copy to IT's queue
SQS	A to-do list	**kept** until a worker takes and deletes them	"prepare a laptop for Arjun"

3 DynamoDB Streams: the table's change log

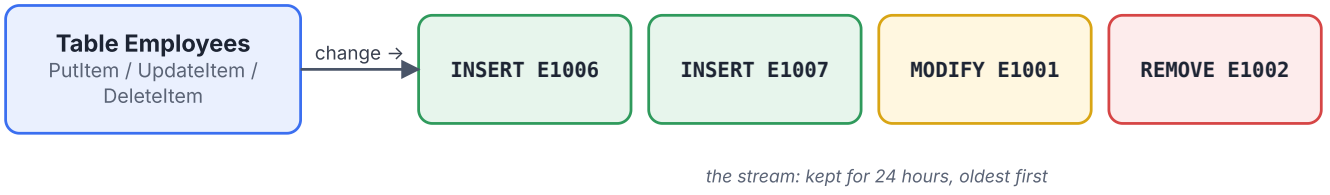


Figure 3. Every change to the table adds one record to its stream, in order.

Turn on a **stream** for a table and DynamoDB writes a record for every change: a new item (**INSERT**), a changed item (**MODIFY**) or a deleted item (**REMOVE**). Your code doesn't send anything — DynamoDB does it.

View type (chosen when you turn it on)	The record contains
Keys only	Just the key, e.g. `emp_id`
New image (our lab)	The whole item **after** the change

View type (chosen when you turn it on)	The record contains
Old image	The whole item **before** the change
New and old images	Both — useful to see what changed

One stream record, as the Lambda receives it:

```
{
  "eventName": "INSERT",
  "eventSource": "aws:dynamodb",
  "awsRegion": "us-east-1",
  "dynamodb": {
    "Keys":     { "emp_id": { "S": "E1006" } },
    "NewImage": { "emp_id":     { "S": "E1006" },
                  "name":       { "S": "Arjun" },
                  "department": { "S": "Finance" },
                  "city":       { "S": "Kolkata" },
                  "leave_balance": { "N": "24" },
                  "rating":     { "N": "4.3" } },
    "StreamViewType": "NEW_IMAGE"
  }
}
```

Values are written in **DynamoDB JSON**: each value is wrapped with its type — `"S"` for text, `"N"` for a number (as text). So `item["name"]["S"]` gives `"Arjun"`.

Good to know	
Only changes **after** you turn it on	Items already in the table are not sent
Records are kept **24 hours**	Then they are gone
No real change → no record	Saving exactly the same item again writes nothing to the stream
In order per item	Changes to E1006 arrive in the order they happened

4 How a Lambda reads a stream

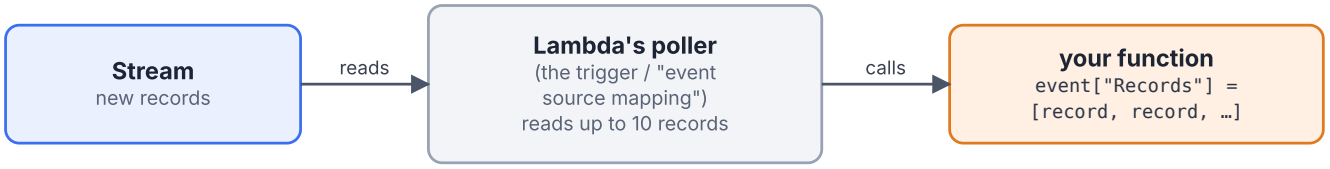


Figure 4. You don't call the function. Lambda keeps reading the stream and calls it with a batch of records.

1. **You add the stream as the function's trigger.** AWS calls this an "event source mapping". You choose a batch size (e.g. 10) and where to start (**Latest** = from now).

2. **Lambda reads the stream for you,** and calls your function with `event["Records"]` — a list of stream records.
3. **Your code loops over the records** and decides what to do with each one, e.g. only act on `INSERT`.

> If the function **fails**, Lambda calls it again with the **same** batch, until it works or the records are 24 hours old. Later changes wait behind it. Keep stream functions simple, and test them.

5 SNS: one message, many receivers

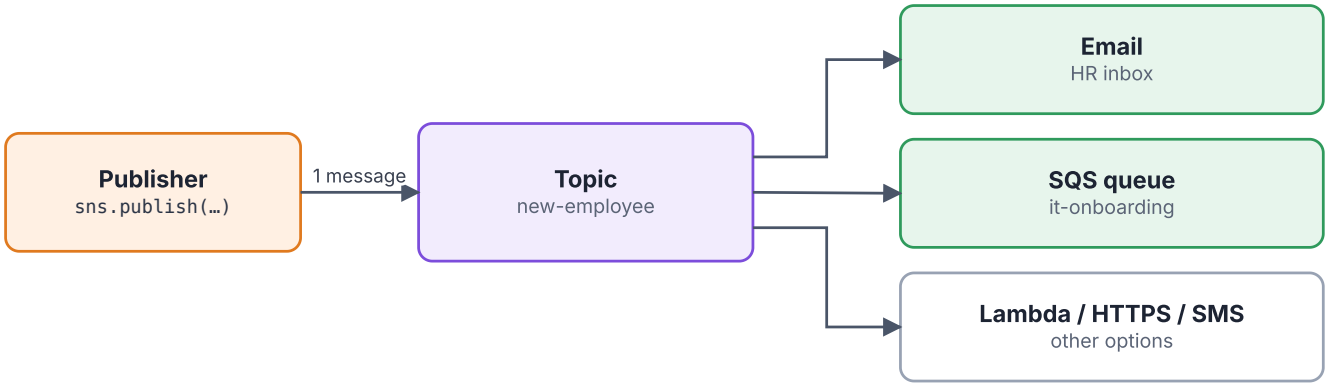


Figure 5. Publish once. SNS pushes a copy to every subscriber of the topic, at the same time.

Word	Meaning
Topic	A named channel, e.g. `new-employee`
Publish	Send one message to the topic
Subscriber	An endpoint that receives a copy: email, SQS queue, Lambda, HTTPS URL, SMS
Fan-out	One message → many copies, one per subscriber

> SNS **pushes** right away and does not keep messages for later. If a subscriber can't take it, SNS retries for a while — but for work that must never be lost, put an SQS queue behind the topic (section 9).

> Email subscribers must **confirm** first: AWS sends a "Subscription Confirmation" email with a link. Until it is clicked, nothing is delivered to that address.

6 What a subscriber receives

The publisher sends a **subject** and a **message** (any text — we send the employee as JSON). Each kind of subscriber gets it differently:

Subscriber	Receives
Email	An email with your subject as the title and your message as the body

Subscriber	Receives
SQS queue	SNS's **envelope**: a JSON document about the delivery, with your message inside `"Message"`


```
{
  "Type": "Notification",
  "MessageId": "3e370cbe-3d35-4ac6-…",
  "TopicArn": "arn:aws:sns:us-east-1:123456789012:new-employee",
  "Subject": "New employee: Arjun",
  "Message": "{\"emp_id\": \"E1006\", \"name\": \"Arjun\", \"department\": \"Finance\", \"city\":
\"Kolkata\"}",
  "Timestamp": "2026-11-05T06:20:11.000Z",
  "Signature": "…", "SigningCertURL": "…", "UnsubscribeURL": "…"
}
```


So the queue's worker reads it in two steps: `envelope = json.loads(body)`, then `employee = json.loads(envelope["Message"])`.

7 SQS: a to-do list that waits

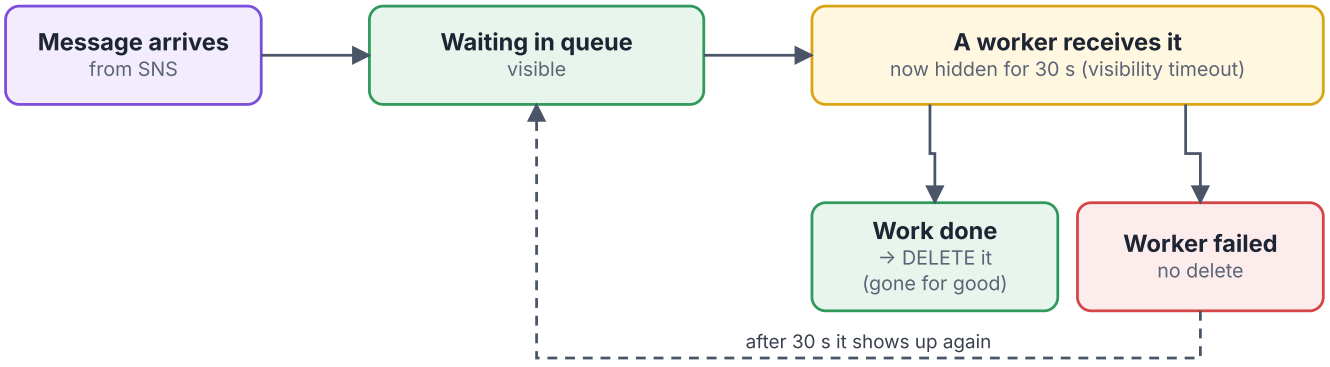


Figure 6. A message stays in the queue until a worker deletes it. While one worker has it, others can't see it.

1. **A message arrives** and waits in the queue (up to 4 days by default, 14 at most).
2. **A worker receives it.** The message is not removed — it is **hidden** from other workers for the **visibility timeout** (30 s by default).
3. **The worker finishes and deletes it.** Now it is gone.
4. **If the worker crashes and never deletes it,** the message becomes visible again and another try can take it. No task is lost.

	Standard queue (our lab)	FIFO queue
Order	Usually in order, not guaranteed	Exactly in order
Duplicates	A message can, rarely, arrive twice	Processed once
Name	any, e.g. `it-onboarding`	must end in `.fifo`

Because a message can rarely come twice, workers should be safe to run twice for the same message — e.g. "prepare laptop for E1006" should not order two laptops.

8 How a Lambda works through a queue

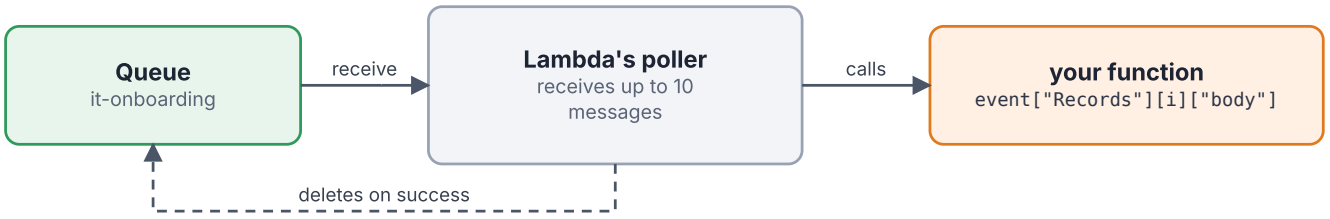


Figure 7. With an SQS trigger, Lambda receives the messages, calls your function, and deletes them if it succeeds.

Your function...	Then Lambda...
returns normally	deletes every message in the batch
raises an error	deletes nothing; the messages reappear after the visibility timeout and are tried again

Without a worker, messages just wait — you can **Poll for messages** in the SQS console and read them yourself. That is the first thing you'll do in the lab.

9 SNS + SQS together: fan-out

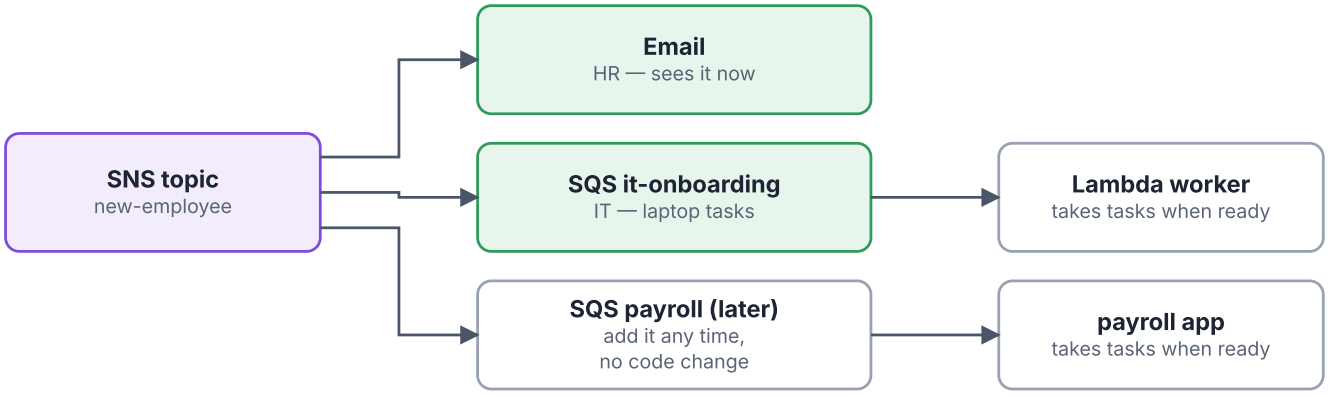


Figure 8. Topic in front, one queue per team behind it. Each team gets its own copy and works at its own speed.

This is the most common pattern with these two services. SNS gives **every** team a copy; a queue per team keeps that copy **safe** until the team's worker is ready.

If...	What happens
IT's worker is down for an hour	IT's messages wait in the queue. HR's email still went out.
Payroll wants the news too	Create a queue and subscribe it to the topic. The publisher doesn't change.

If...	What happens
HR unsubscribes the email	IT's queue still gets every message.

10 The permissions this needs

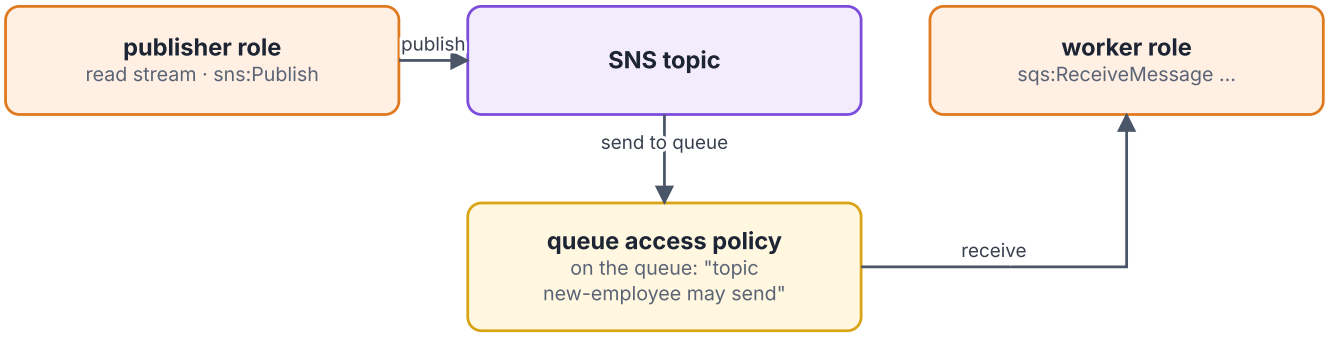


Figure 9. Three permissions make the chain work. Two are on roles; one is on the queue itself.

Who	Needs	Where it is written
Publisher Lambda	`dynamodb:DescribeStream`, `GetRecords`, `GetShardIterator`, `ListStreams` on the stream; `sns:Publish` on the topic; logs	Its role's policy (identity-based)
SNS topic	`sqs:SendMessage` to the queue	The **queue's access policy** (resource-based)
Worker Lambda	`sqs:ReceiveMessage`, `DeleteMessage`, `GetQueueAttributes` on the queue; logs	Its role's policy (identity-based)

The IAM guide put **resource-based policies** in "skip for now". Here is your first one: it sits **on the queue** and says who may send to it.

```
{
  "Effect": "Allow",
  "Principal": { "Service": "sns.amazonaws.com" },
  "Action": "sqs:SendMessage",
  "Resource": "arn:aws:sqs:us-east-1:123456789012:it-onboarding",
  "Condition": { "ArnEquals": { "aws:SourceArn": "arn:aws:sns:us-east-1:123456789012:new-employee"
} }
}
```

Read it as: SNS may send to this queue — **but only** when the message comes from the topic `new-employee`. The `Condition` stops other topics from filling your queue. In the console, the button **Subscribe to Amazon SNS topic** writes this for you.

11 When to use which

You want to…	Use	Example
React when data in a DynamoDB table changes	**DynamoDB Streams** + Lambda	A new employee row → start onboarding
Tell several receivers at once	**SNS** topic	Email HR and notify IT about the same event
Hand work to a worker that may be slow or down	**SQS** queue	Laptop requests, sending invoices, resizing images
Both: everyone gets a copy, nobody loses one	**SNS → SQS** (fan-out)	One queue per team behind one topic
Strict order, no duplicates	SQS **FIFO** queue	Bank account updates for one customer

> Short version: **stream** = "what changed", **SNS** = "tell everyone now", **SQS** = "do this later, and don't lose it".

12 The lab in one picture

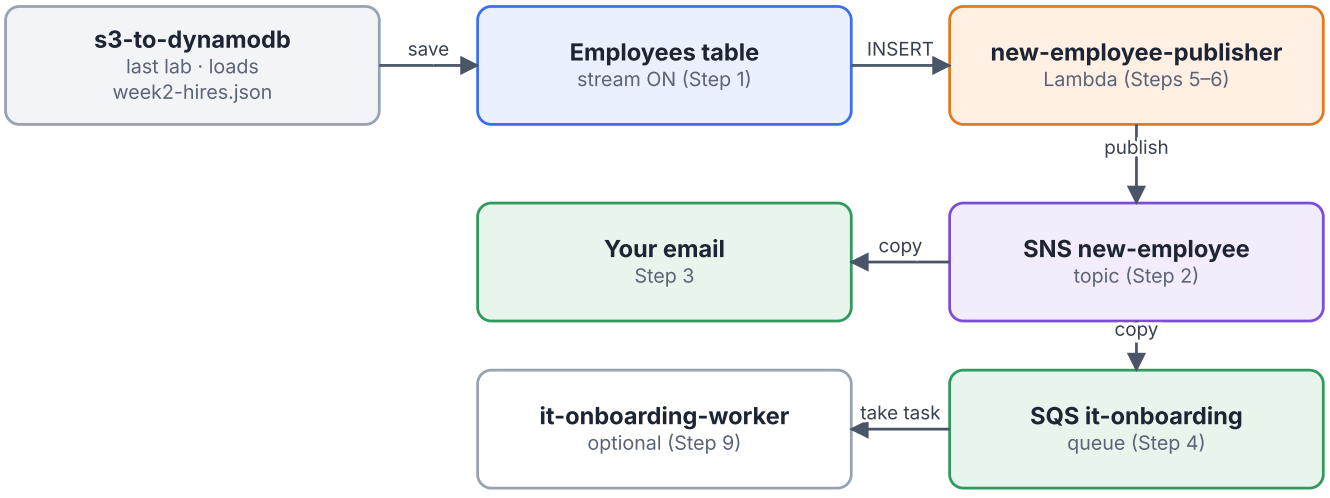


Figure 10. The lab. Each box is one step.

In the lab you load two new employees with last lab's function. Without pressing anything else, you get two emails and two tasks in IT's queue — and, optionally, a worker that prints "prepare a laptop for Arjun".

13 What you can skip for now

Topic	In one line
Dead-letter queue (DLQ)	A second queue where messages go after failing too many times, so they stop retrying

Topic	In one line
Partial batch failures	Telling Lambda which messages in a batch failed, so only those are retried
SNS filter policies	A subscriber gets only the messages it cares about (e.g. only department IT)
FIFO details	Message groups and de-duplication IDs
Long polling, batch windows	Tuning how Lambda reads queues and streams
Kinesis, EventBridge	Other ways to stream and route events (EventBridge comes in Module 8)

14 Words to know

Word	Meaning
Decoupling	The sender doesn't call the receivers; a message sits in between
Stream record	One change in a DynamoDB table: INSERT, MODIFY or REMOVE
Event source mapping (trigger)	Lambda reading a stream or queue and calling your function
Topic	SNS channel that pushes a copy to every subscriber
Subscription	One receiver of a topic (email, queue, Lambda ...)
Envelope	SNS's JSON wrapper around your message, as SQS receives it
Queue	SQS list of messages waiting for a worker
Visibility timeout	How long a received message stays hidden before it reappears
Fan-out	One message to many receivers
Resource-based policy	A policy on the resource itself (here: the queue) saying who may use it

15 Key takeaways

1. A message in the middle lets services work on their own: a slow or broken receiver doesn't break the sender.
2. A DynamoDB stream records every change (INSERT / MODIFY / REMOVE) for 24 hours; Lambda can react to it.
3. SNS pushes one message to every subscriber at once, and doesn't keep it.
4. SQS keeps a message until a worker deletes it; if the worker fails, the message comes back.
5. SNS + one SQS queue per team = everyone gets a copy and nobody loses one.
6. Roles give the Lambdas their permissions; the queue's own access policy lets the topic send to it.